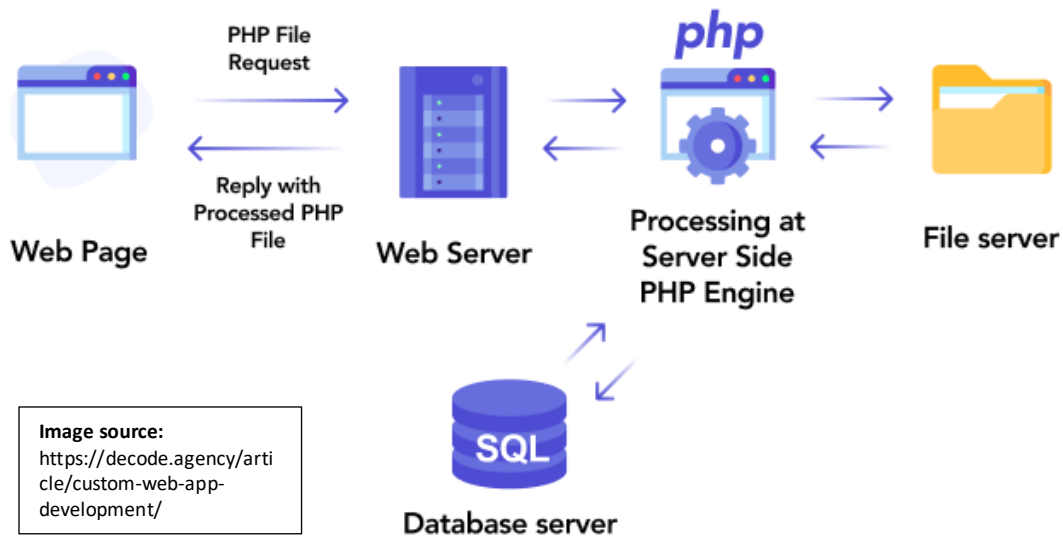


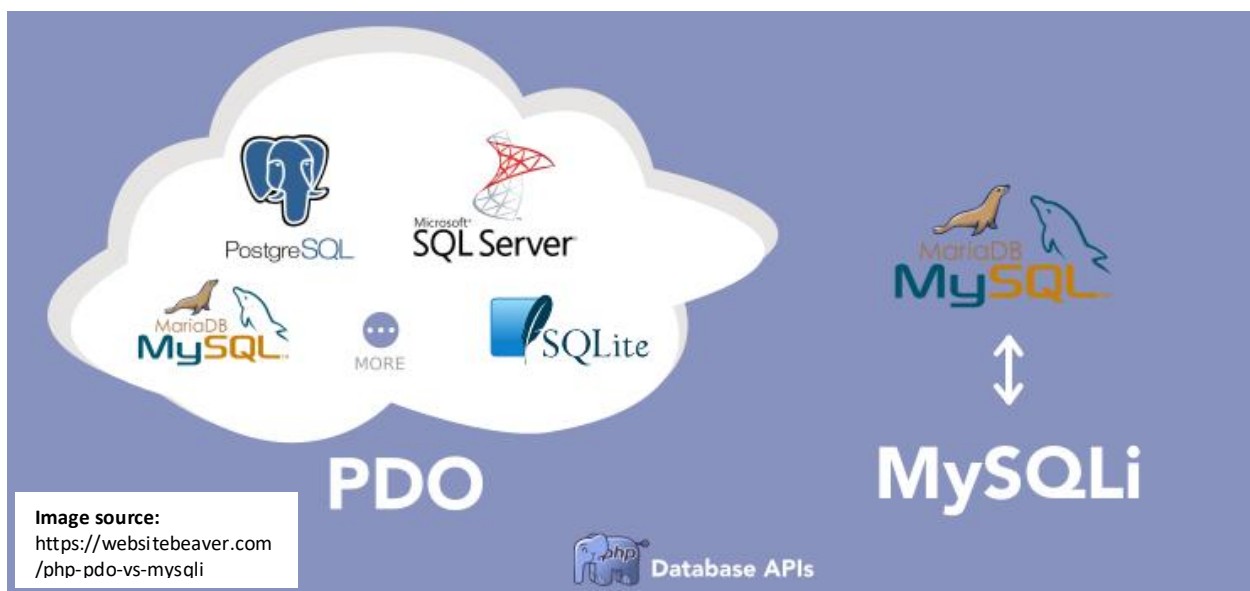
## Chapter 4

### PHP-MySQL INTEGRATION

#### 4.1. Connecting PHP with MySQL



PHP provides two primary ways to interact with MySQL databases: MySQLi (MySQL Improved) and PDO (PHP Data Objects). Both offer secure and efficient ways to execute database queries, but they differ in functionality and use cases.



### MySQLi vs. PDO (Pros & Cons)

| MySQLi                       | PDO                                                                                                                           |
|------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Only MySQL                   | Supports multiple databases (MySQL, PostgreSQL, SQLite, etc.)<br>More flexible (switching databases requires minimal changes) |
| Supports prepared statements | Fully supports prepared statements                                                                                            |
| Both Procedural and OOP      | Only OOP                                                                                                                      |
| Slightly faster for MySQL    | Minor performance difference                                                                                                  |

### Connecting with MySQLi

MySQLi offers two modes for connecting to a database, Procedural Style and Object-Oriented Style.

#### MySQLi Procedural Connection

```
<?php
    $server = "localhost";
    $username = "root";
    $password = "";
    $database = "test_db";

    // Create connection
    $conn = mysqli_connect($server, $username, $password, $database);

    // Check connection
    if (!$conn) {
        die("Connection failed: " . mysqli_connect_error());
    }

    echo "Connected successfully";

?>
```

The `mysqli_connect()` function establishes a connection between PHP and a MySQL database. If the connection fails, the `mysqli_connect_error()` function returns an error message describing the issue, allowing developers to handle connection failures effectively.

#### MySQLi Object-Oriented Connection

```
<?php
    $server = "localhost";
    $username = "root";
    $password = "";
    $database = "test_db";

    // Create connection
    $conn = new mysqli($server, $username, $password, $database);

    // Check connection
    if ($conn->connect_error) {
        die("Connection failed: " . $conn->connect_error);
    }
```

```

        echo "Connected successfully";
    ?>

```

The `$conn = new mysqli(...)` statement initializes a new MySQLi object to establish a connection with the MySQL database. If the connection fails, the `$conn->connect_error` property contains the error message, allowing developers to detect and handle connection issues.

## 4.2. Connecting with PDO

PDO provides a more secure and flexible way to interact with databases.

### PDO Connection

```

<?php
    $server = "localhost";
    $username = "root";
    $password = "";
    $database = "test_db";

    try {
        // Create a new PDO instance
        $conn = new PDO("mysql:host=$server;dbname=$database", $username,
            $password);

        // Set error mode to exceptions
        $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

        echo "Connected successfully";
    } catch (PDOException $e) {
        echo "Connection failed: " . $e->getMessage();
    }

    ?>

```

The `new PDO("mysql:host=$server;dbname=$database", $username, $password)` statement creates a connection to the MySQL database using the PDO extension. The `setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION)` method ensures that errors are thrown as exceptions, making them easier to handle. If a connection error occurs, the `catch (PDOException $e)` block catches the exception and displays an appropriate error message.

### DSN (Data Source Name) in PDO

A DSN (Data Source Name) defines the database connection settings in PDO. The general format is:

```
dsn = "mysql:host=HOSTNAME;dbname=DATABASE_NAME;charset=UTF8"
```

Example:

```

$dsn = "mysql:host=localhost;dbname=test_db;charset=utf8";
$conn = new PDO($dsn, $username, $password);

```

## 4.3. Creating Database

You can create a new database if it does not already exist.

### Using MySQLi (Procedural)

```
<?php
    $servername = "localhost";
    $username = "root";
    $password = "";

    // Create connection
    $conn = mysqli_connect($servername, $username, $password);

    // Check connection
    if (!$conn) {
        die("Connection failed: " . mysqli_connect_error());
    }

    // Create database
    $sql = "CREATE DATABASE test_db";
    if (mysqli_query($conn, $sql)) {
        echo "Database created successfully";
    } else {
        echo "Error creating database: " . mysqli_error($conn);
    }

    mysqli_close($conn);

?>
```

### Using MySQLi (Object-Oriented)

```
<?php
    $servername = "localhost";
    $username = "root";
    $password = "";

    // Create connection
    $conn = new mysqli($servername, $username, $password);

    // Check connection
    if ($conn->connect_error) {
        die("Connection failed: " . $conn->connect_error);
    }

    // Create database
    $sql = "CREATE DATABASE test_db";
    if ($conn->query($sql) === TRUE) {
        echo "Database created successfully";
    } else {
        echo "Error creating database: " . $conn->error;
    }

    $conn->close();
```

```
?>
```

### Using PDO (PHP Data Objects)

```
<?php
    $servername = "localhost";
    $username = "root";
    $password = "";

    try {
        $conn = new PDO("mysql:host=$servername", $username, $password);
        $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

        $sql = "CREATE DATABASE test_db";
        $conn->exec($sql);
        echo "Database created successfully";
    } catch (PDOException $e) {
        echo "Error creating database: " . $e->getMessage();
    }

    $conn = null;

?>
```

## 4.4. Create Table

### Using MySQLi (Procedural)

```
<?php
    $servername = "localhost";
    $username = "root";
    $password = "";
    $dbname = "test_db";

    // Create connection
    $conn = mysqli_connect($servername, $username, $password, $dbname);

    // Check connection
    if (!$conn) {
        die("Connection failed: " . mysqli_connect_error());
    }

    // Create table
    $sql = "CREATE TABLE IF NOT EXISTS Users (
        id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
        name VARCHAR(50) NOT NULL,
        email VARCHAR(50) NOT NULL
    )";

    if (mysqli_query($conn, $sql)) {
        echo "Table Users created successfully";
    } else {
        echo "Error creating table: " . mysqli_error($conn);
    }

    mysqli_close($conn);

?>
```

**Using MySQLi (Object-Oriented)**

```

<?php
    $conn = new mysqli($servername, $username, $password, $dbname);

    if ($conn->connect_error) {
        die("Connection failed: " . $conn->connect_error);
    }

    $sql = "CREATE TABLE IF NOT EXISTS Users (
        id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
        name VARCHAR(50) NOT NULL,
        email VARCHAR(50) NOT NULL
    )";

    if ($conn->query($sql) === TRUE) {
        echo "Table Users created successfully";
    } else {
        echo "Error creating table: " . $conn->error;
    }

    $conn->close();

?>

```

**Using PDO (PHP Data Objects)**

```

<?php
    try {
        $conn = new PDO("mysql:host=$servername;dbname=$dbname",
            $username, $password);
        $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

        $sql = "CREATE TABLE IF NOT EXISTS Users (
            id INT(6) UNSIGNED AUTO_INCREMENT PRIMARY KEY,
            name VARCHAR(50) NOT NULL,
            email VARCHAR(50) NOT NULL
        )";

        $conn->exec($sql);
        echo "Table Users created successfully";
    } catch (PDOException $e) {
        echo "Error creating table: " . $e->getMessage();
    }

    $conn = null;

?>

```

## 4.5. CRUD Operations

### CREATE (INSERT Data)

**Using MySQLi (Procedural Approach):** The procedural approach uses `mysqli_query()` to execute SQL queries.

```
<?php
// Database connection
$conn = mysqli_connect("localhost", "root", "", "test_db");

// Check connection
if (!$conn) {
    die("Connection failed: " . mysqli_connect_error());
}

// Insert query
$sql = "INSERT INTO users (name, email) VALUES ('John Doe',
'john@example.com')";

// Execute query
if (mysqli_query($conn, $sql)) {
    echo "New record created successfully";
} else {
    echo "Error: " . mysqli_error($conn);
}

// Close connection
mysqli_close($conn);

?>
```

**Using MySQLi (Object-Oriented Approach):** The OOP approach uses the `mysqli` class to handle queries.

```
<?php
// Database connection
$conn = new mysqli("localhost", "root", "", "test_db");

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

// Insert query
$sql = "INSERT INTO users (name, email) VALUES ('Jane Doe',
'jane@example.com')";

// Execute query
if ($conn->query($sql) === TRUE) {
    echo "New record created successfully";
} else {
    echo "Error: " . $conn->error;
}

$conn->close();

?>
```

**Using PDO (Prepared Statements):** Using prepared statements improves security by preventing SQL injection.

```
<?php
    try {
        // Database connection
        $conn = new PDO("mysql:host=localhost;dbname=test_db", "root",
            "");
        $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

        // Prepare SQL statement
        $stmt = $conn->prepare("INSERT INTO users (name, email) VALUES
            (:name, :email)");

        // Bind parameters
        $stmt->bindParam(':name', $name);
        $stmt->bindParam(':email', $email);

        // Assign values and execute
        $name = "Alice Doe";
        $email = "alice@example.com";
        $stmt->execute();

        echo "New record created successfully";
    } catch (PDOException $e) {
        echo "Error: " . $e->getMessage();
    }

    // Close connection
    $conn = null;

?>
```

## READ (SELECT Data)

### Fetching Data Using MySQLi (Procedural)

```
<?php
    $conn = mysqli_connect("localhost", "root", "", "test_db");

    $sql = "SELECT id, name, email FROM users";
    $result = mysqli_query($conn, $sql);

    if (mysqli_num_rows($result) > 0) {
        while ($row = mysqli_fetch_assoc($result)) {
            echo "ID: " . $row["id"] . " - Name: " . $row["name"] . " -
            Email: " . $row["email"] . "<br>";
        }
    } else {
        echo "No results found";
    }

    mysqli_close($conn);

?>
```



**Fetching Data Using MySQLi (OOP)**

```

<?php
    $conn = new mysqli("localhost", "root", "", "test_db");

    $sql = "SELECT id, name, email FROM users";
    $result = $conn->query($sql);

    if ($result->num_rows > 0) {
        while ($row = $result->fetch_object()) {
            echo "ID: " . $row->id . " - Name: " . $row->name . " - Email: " . $row->email . "<br>";
        }
    } else {
        echo "No results found";
    }

    $conn->close();
?>

```

**Fetching Data Using PDO**

```

<?php
    $conn = new PDO("mysql:host=localhost;dbname=test_db", "root", "");
    $conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $stmt = $conn->prepare("SELECT id, name, email FROM users");
    $stmt->execute();
    $result = $stmt->fetchAll(PDO::FETCH_ASSOC);

    foreach ($result as $row) {
        echo "ID: " . $row["id"] . " - Name: " . $row["name"] . " - Email: " . $row["email"] . "<br>";
    }

    $conn = null;
?>

```

**UPDATE (UPDATE Data)****Using MySQLi (Procedural)**

```

<?php
    $conn = mysqli_connect("localhost", "root", "", "test_db");

    $sql = "UPDATE users SET email='newemail@example.com' WHERE id=1";

    if (mysqli_query($conn, $sql)) {
        echo "Record updated successfully";
    } else {
        echo "Error updating record: " . mysqli_error($conn);
    }

    mysqli_close($conn);
?>

```

**Using PDO (Prepared Statement)**

```

<?php
$conn = new PDO("mysql:host=localhost;dbname=test_db", "root", "");
$conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

$stmt = $conn->prepare("UPDATE users SET email=:email WHERE id=:id");
$stmt->bindParam(':email', $email);
$stmt->bindParam(':id', $id);

$email = "updated@example.com";
$id = 1;
$stmt->execute();

echo "Record updated successfully";
$conn = null;
?>

```

**DELETE (DELETE Data)****Using MySQLi (Procedural)**

```

<?php
$conn = mysqli_connect("localhost", "root", "", "test_db");

$sql = "DELETE FROM users WHERE id=1";

if (mysqli_query($conn, $sql)) {
    echo "Record deleted successfully";
} else {
    echo "Error deleting record: " . mysqli_error($conn);
}

mysqli_close($conn);
?>

```

## Using PDO (Prepared Statement)

```
<?php
$conn = new PDO("mysql:host=localhost;dbname=test_db", "root", "");
$conn->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

$stmt = $conn->prepare("DELETE FROM users WHERE id=:id");
$stmt->bindParam(':id', $id);

$id = 1;
$stmt->execute();

echo "Record deleted successfully";
$conn = null;

?>
```

## 4.6. Prepared Statements & Security

Prepared statements are a feature of many database systems that allow you to execute the same SQL query multiple times with different inputs. They separate the SQL structure from the data, which brings several benefits:

- **Security:** Prevents SQL injection by ensuring user inputs are treated strictly as data, not part of the SQL query.
- **Performance:** The database parses and compiles the query only once, improving efficiency when running the query multiple times.
- **Readability & Maintenance:** The code is cleaner and more manageable by avoiding direct string manipulation.

### SQL Injection Prevention

SQL injection is a common web security vulnerability where an attacker manipulates an SQL query by inserting malicious SQL code. Example of a vulnerable query:

```
$username = $_POST['username'];
$password = $_POST['password'];

$query = "SELECT * FROM users WHERE username = '$username' AND password = '$password'";
```

If a user inputs `admin' --` as the username, the query becomes:

```
SELECT * FROM users WHERE username = 'admin' -- ' AND password = 'password'
```

This logs in without a password check! Prepared statements mitigate this by treating user input strictly as data.

### Using `bind_param()` with MySQLi

MySQLi offers the `bind_param()` function to bind user inputs securely to placeholders.

#### Example:

```
$stmt = $mysqli->prepare("SELECT * FROM users WHERE username = ? AND password = ?");
$stmt->bind_param("ss", $username, $password);
$stmt->execute();
$result = $stmt->get_result();
```

- `?`: Placeholder for data.
- `"ss"`: Specifies the types (`s` for string) corresponding to each placeholder.

| Type Code | Description                                 |
|-----------|---------------------------------------------|
| i         | Integer                                     |
| d         | Double (floating point)                     |
| s         | String                                      |
| b         | Blob (binary data, must be sent in packets) |

### Using `bindValue()` and `bindParam()` with PDO

PDO (PHP Data Objects) provides a more flexible approach

#### Example - `bindValue`

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = :username AND password = :password");
$stmt->bindValue(':username', $username, PDO::PARAM_STR);
$stmt->bindValue(':password', $password, PDO::PARAM_STR);
$stmt->execute();
```

#### Example - `bindParam`

```
$stmt = $pdo->prepare("SELECT * FROM users WHERE username = :username AND password = :password");
$stmt->bindParam(':username', $username, PDO::PARAM_STR);
$stmt->bindParam(':password', $password, PDO::PARAM_STR);

// Set values to the variables
$username = "john_doe";
$password = "securepassword";

// Now execute the statement
$stmt->execute();
```

- `:username` and `:password` are named placeholders.
- `bindValue()` binds the value immediately.
- `bindParam()` binds a variable by reference, useful if the variable's value changes before execution.

`bindValue()` binds the data directly at the time of execution, meaning the value remains unchanged even if the original variable is modified later. In contrast, `bindParam()` binds a variable by reference, allowing it to reflect the latest value of the variable when the statement is executed.

| Data Type          | PDO Constant                 | Description                                                     |
|--------------------|------------------------------|-----------------------------------------------------------------|
| String             | <code>PDO::PARAM_STR</code>  | For any string or textual data.                                 |
| Integer            | <code>PDO::PARAM_INT</code>  | For integers (whole numbers).                                   |
| Boolean            | <code>PDO::PARAM_BOOL</code> | For boolean values ( <code>true</code> or <code>false</code> ). |
| NULL               | <code>PDO::PARAM_NULL</code> | For NULL values.                                                |
| Large Object (LOB) | <code>PDO::PARAM_LOB</code>  | For large binary data (e.g., images, files).                    |

## 4.7. Handling Errors & Exceptions

### MySQLi: `mysqli_error()` & `errno`

When working with MySQLi in PHP, errors can occur, from invalid SQL queries to connection issues. To handle these, `mysqli_error()` and `mysqli_errno()` functions are essential.

#### Example:

```
$conn = new mysqli("localhost", "username", "password", "database");

if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}

$sql = "SELECT * FROM non_existing_table";
$result = $conn->query($sql);

if (!$result) {
    echo "Error: " . $conn->error . " (" . $conn->errno . ")";
}
```

The `mysqli_error()` function returns a string description of the last error that occurred in the most recent MySQL operation, making it useful for understanding what went wrong during database interactions. Similarly, `mysqli_errno()` returns the corresponding error code, which can be helpful for more detailed debugging or when implementing custom error handling based on specific error numbers.

### PDO: try-catch with PDOException

PDO (PHP Data Objects) provides a more elegant way to handle errors using exceptions.

#### Example:

```
try {
    $pdo = new PDO("mysql:host=localhost;dbname=database", "username",
"password");
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);

    $sql = "SELECT * FROM non_existing_table";
    $pdo->query($sql);
} catch (PDOException $e) {
    echo "Error: " . $e->getMessage();
}
```

`PDO::ERRMODE_EXCEPTION` triggers an exception whenever a database error occurs, allowing the program to handle errors more cleanly and avoid silent failures. The `catch` block then captures the thrown exception and provides a detailed error message, making it easier to debug and understand what went wrong during the database operation.

### Logging vs. Displaying Errors

When handling errors, it's important to decide whether to display the error to users or log it for developers.

#### Displaying Errors

This approach is helpful during development because it immediately shows errors on the web page, making it easier to spot and fix issues. However, it's dangerous in a production environment since the error messages might reveal sensitive information, like file paths or database details, which could be exploited by attackers. To enable error display, you can use:

```
ini_set('display_errors', 1);
error_reporting(E_ALL);
```

#### Logging Errors

In production, it's safer to log errors instead of displaying them to users. This way, you still keep track of issues without exposing sensitive data publicly. You can enable error logging like this:

```
ini_set('log_errors', 1);
ini_set('error_log', 'errors.log');
error_log("Error occurred on " . date('Y-m-d H:i:s'));
```

The `ini_set('log_errors', 1)` directive ensures PHP logs errors, while `ini_set('error_log', 'errors.log')` specifies the log file to store them. Additionally, `error_log()` allows you to write

custom error messages, including timestamps for easier tracking. This setup keeps your application secure and helps with debugging behind the scenes.

## 4.8. Database Environment, Transactions and Performance

### Environment Variables for DB Credentials (.env)

Storing database credentials directly in your PHP files is unsafe. A better approach is to use environment variables, typically stored in a `.env` file (especially common in frameworks like Laravel).

#### Example `.env` file:

```
DB_HOST=localhost
DB_NAME=database
DB_USER=username
DB_PASS=password
```

#### PHP Configuration:

```
$host = getenv('DB_HOST');
$db = getenv('DB_NAME');
$user = getenv('DB_USER');
$pass = getenv('DB_PASS');

try {
    $pdo = new PDO("mysql:host=$host;dbname=$db", $user, $pass);
    $pdo->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
} catch (PDOException $e) {
    die("Connection failed: " . $e->getMessage());
}
```

### Using Transactions for Data Integrity (`commit()`, `rollback()`)

A transaction ensures a group of SQL statements either all succeed or all fail; maintaining data consistency.

#### Example:

```
try {
    $pdo->beginTransaction();

    $pdo->exec("INSERT INTO accounts (user, balance) VALUES ('John',
1000)");
    $pdo->exec("UPDATE accounts SET balance = balance - 500 WHERE user =
'John'");

    $pdo->commit();
} catch (PDOException $e) {
    $pdo->rollback();
    echo "Transaction failed: " . $e->getMessage();
}
```

- **`beginTransaction()`** starts the transaction.
- **`commit()`** saves the changes if everything is successful.

- `rollback()` reverts all changes if an error occurs.

## Performance: Indexes & Query Optimization

As your data grows, performance matters! Here's how to improve it:

1. **Indexes:** Indexes speed up searches by creating a lookup table for faster access.

### Example:

```
CREATE INDEX idx_username ON users(username);
```

This helps queries like:

```
SELECT * FROM users WHERE username = 'JohnDoe';
```

run significantly faster.

**Tip:** Don't overuse indexes — they speed up reads but slow down inserts/updates.

### 2. Query Optimization:

- Avoid **SELECT \*** Select only the columns you need.
- Use JOINS efficiently, Fetch related data in a single query instead of multiple queries.
- Limit results:  

```
SELECT * FROM orders ORDER BY date DESC LIMIT 10;
```
- Cache results for data that does not change often, caching reduces repetitive queries.